

# Tài liệu Hướng dẫn Thiết kế Bộ điều khiển DDWMR

## Trajectory Tracking trên phần cứng ESP32

TS. Vũ Trí Viễn

Ngày 15 tháng 4 năm 2026

## Mục lục

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Tổng quan Bài toán Thực tế</b>                       | <b>2</b>  |
| <b>2</b> | <b>Nền tảng Toán học và Động học</b>                    | <b>2</b>  |
| 2.1      | Mô hình Động học DDWMR . . . . .                        | 2         |
| 2.2      | Tạo Quỹ đạo Tham chiếu . . . . .                        | 2         |
| <b>3</b> | <b>Lý thuyết Điều khiển Phân tầng</b>                   | <b>2</b>  |
| 3.1      | Điều khiển Quỹ đạo: Backstepping (High-level) . . . . . | 2         |
| <b>4</b> | <b>Điều khiển Động cơ (Low-level): Khâu PID</b>         | <b>5</b>  |
| 4.1      | Luật điều khiển PID . . . . .                           | 5         |
| 4.2      | Chống bão hòa tích phân (Anti-windup) . . . . .         | 5         |
| <b>5</b> | <b>Định vị và Dung hợp Cảm biến (Sensor Fusion)</b>     | <b>5</b>  |
| 5.1      | Tính toán Odometry (Định vị tương đối) . . . . .        | 5         |
| 5.2      | Dung hợp cảm biến với IMU MPU6050 . . . . .             | 6         |
| <b>6</b> | <b>Thực thi Phần cứng trên ESP32</b>                    | <b>6</b>  |
| <b>7</b> | <b>Kết luận và Lưu ý Triển khai</b>                     | <b>10</b> |

# 1 Tổng quan Bài toán Thực tế

Bài toán đặt ra là điều khiển một Robot di động hai bánh chủ động (Differential Drive Wheeled Mobile Robot - DDWMR) bám theo một quỹ đạo cho trước. Trong môi trường thực tế, hệ thống phải đối mặt với các vấn đề:

- **Động học phi tuyến:** Mối quan hệ giữa vận tốc động cơ và tọa độ không gian của robot là phi tuyến.
- **Nhiều hệ thống và đo lường:** Cảm biến encoder có thể bị sai số do trượt bánh xe, trong khi động cơ chịu ảnh hưởng của ma sát và biến thiên tải trọng.

Để giải quyết, tài liệu này đề xuất cấu trúc điều khiển phân tầng (Hierarchical Control): Tầng cao sử dụng thuật toán **Backstepping** để triệt tiêu sai số quỹ đạo; Tầng thấp sử dụng **LQG** để điều khiển momen động cơ; Đồng thời sử dụng **Odometry** và **EKF** kết hợp IMU để định vị chính xác.

## 2 Nền tảng Toán học và Động học

### 2.1 Mô hình Động học DDWMR

Phương trình trạng thái của Robot được mô tả bằng vector  $q = [x, y, \theta]^T$ :

$$\dot{q} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (1)$$

Trong đó,  $v$  và  $\omega$  lần lượt là vận tốc dài và vận tốc góc của robot.

### 2.2 Tạo Quỹ đạo Tham chiếu

Để bộ điều khiển bám hoạt động mượt mà, cần cung cấp tọa độ mong muốn  $(x_d, y_d)$  cùng vận tốc tham chiếu tương ứng  $v_d, \omega_d$ . Các giá trị này được tính từ đạo hàm của phương trình quỹ đạo:

$$v_d = \sqrt{\dot{x}_d^2 + \dot{y}_d^2}, \quad \theta_d = \text{atan2}(\dot{y}_d, \dot{x}_d) \quad (2)$$

$$\omega_d = \frac{\dot{x}_d \ddot{y}_d - \dot{y}_d \ddot{x}_d}{\dot{x}_d^2 + \dot{y}_d^2} \quad (3)$$

Việc tính toán sẵn  $\omega_d$  giúp hệ thống triệt tiêu sai số hướng  $e_\theta$  nhanh hơn thông qua thuật ngữ feedforward. Tại các điểm vận tốc bằng 0, giá trị  $\omega_d$  có thể bị bất định, do đó cần thêm một lượng nhỏ  $\epsilon$  vào mẫu số khi lập trình.

## 3 Lý thuyết Điều khiển Phân tầng

### 3.1 Điều khiển Quỹ đạo: Backstepping (High-level)

Sai số bám trong hệ tọa độ địa phương được định nghĩa là:

$$e = \begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{bmatrix} \quad (4)$$

Sử dụng hàm Lyapunov  $V = \frac{1}{2}(e_x^2 + e_y^2) + \frac{1 - \cos e_\theta}{k_y}$ , luật điều khiển đề xuất để  $\dot{V}$  xác định âm là:

$$v = v_d \cos e_\theta + k_x e_x \quad (5)$$

$$\omega = \omega_d + v_d(k_y e_y + k_\theta \sin e_\theta) \quad (6)$$

Chi tiết thiết kế bộ điều khiển như sau

### Bước 1: Mô hình hóa hệ thống và quỹ đạo tham chiếu

Đầu tiên, ta cần có phương trình toán học mô tả cách robot di chuyển và cách quỹ đạo mẫu di chuyển.

- **Mô hình động học của Robot:**

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (7)$$

- **Quỹ đạo tham chiếu (Reference):** (Ví dụ như một con robot ảo chạy chuẩn trên đường)

$$\begin{bmatrix} \dot{x}_r \\ \dot{y}_r \\ \dot{\theta}_r \end{bmatrix} = \begin{bmatrix} \cos \theta_r & 0 \\ \sin \theta_r & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_r \\ \omega_r \end{bmatrix} \quad (8)$$

### Bước 2: Định nghĩa sai số bám (Tracking Error)

Ta không tính sai số trên hệ tọa độ toàn cục (Global) mà phải nhân với ma trận quay (Rotation Matrix) để chuyển sai số về hệ tọa độ cục bộ gắn trên robot (Local Frame). Điều này giúp bộ điều khiển biết chính xác mục tiêu đang nằm ở hướng nào so với mũi robot.

$$\begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix} \quad (9)$$

### Bước 3: Tìm động lực học của sai số (Error Dynamics)

Lấy đạo hàm hai vế của phương trình sai số ở Bước 2 theo thời gian, kết hợp với các phương trình ở Bước 1, ta thu được phương trình vi phân mô tả sự biến thiên của sai số:

$$\begin{cases} \dot{e}_x = \omega e_y - v + v_r \cos e_\theta \\ \dot{e}_y = -\omega e_x + v_r \sin e_\theta \\ \dot{e}_\theta = \omega_r - \omega \end{cases} \quad (10)$$

*Nhiệm vụ bây giờ là phải tìm các giá trị đầu vào  $v$  và  $\omega$  sao cho  $e_x, e_y, e_\theta$  tiến về 0 khi thời gian  $t \rightarrow \infty$ .*

#### Bước 4: Lựa chọn hàm ứng viên Lyapunov

Để hệ thống ổn định, ta chọn một hàm năng lượng giả định  $V \geq 0$ . Hàm này thường được chọn dưới dạng:

$$V = \frac{1}{2}e_x^2 + \frac{1}{2}e_y^2 + \frac{1 - \cos e_\theta}{k_y} \quad (11)$$

Trong đó  $k_y > 0$  là hệ số thiết kế. Lý do dùng  $(1 - \cos e_\theta)$  là vì khi  $e_\theta = 0$  thì phần này bằng 0, và nó luôn dương khi  $e_\theta \neq 0$  trong khoảng hẹp.

#### Bước 5: Thiết kế luật điều khiển để $\dot{V} \leq 0$

Lấy đạo hàm của hàm  $V$  theo thời gian:

$$\dot{V} = e_x \dot{e}_x + e_y \dot{e}_y + \frac{\sin e_\theta}{k_y} \dot{e}_\theta \quad (12)$$

Thay các phương trình động lực học sai số (ở Bước 3) vào  $\dot{V}$ :

$$\dot{V} = e_x(\omega e_y - v + v_r \cos e_\theta) + e_y(-\omega e_x + v_r \sin e_\theta) + \frac{\sin e_\theta}{k_y}(\omega_r - \omega) \quad (13)$$

Khai triển và triệt tiêu cặp  $e_x \omega e_y - e_y \omega e_x = 0$ , ta gom lại được:

$$\dot{V} = e_x(-v + v_r \cos e_\theta) + e_y v_r \sin e_\theta + \frac{\sin e_\theta}{k_y}(\omega_r - \omega) \quad (14)$$

Bây giờ, ta chọn luật điều khiển  $(v, \omega)$  sao cho triệt tiêu các thành phần gây mất ổn định và làm  $\dot{V}$  mang dấu âm:

**Chọn vận tốc dài  $v$**

$$v = v_r \cos e_\theta + k_x e_x \quad (\text{với } k_x > 0) \quad (15)$$

Thay vào  $\dot{V}$ , phần đầu tiên sẽ biến thành:  $e_x(-k_x e_x) = -k_x e_x^2$ .

**Chọn vận tốc góc  $\omega$**  Lúc này  $\dot{V}$  còn lại:

$$\dot{V} = -k_x e_x^2 + e_y v_r \sin e_\theta + \frac{\sin e_\theta}{k_y}(\omega_r - \omega) \quad (16)$$

Rút  $\frac{\sin e_\theta}{k_y}$  làm nhân tử chung cho 2 vế sau:

$$\dot{V} = -k_x e_x^2 + \frac{\sin e_\theta}{k_y} (k_y v_r e_y + \omega_r - \omega) \quad (17)$$

Để triệt tiêu cụm trong ngoặc và tạo ra số âm, ta thiết kế:

$$\omega = \omega_r + k_y v_r e_y + k_\theta \sin e_\theta \quad (\text{với } k_\theta > 0) \quad (18)$$

**Kết luận** Thay lại luật điều khiển  $\omega$  vào  $\dot{V}$ , ta được kết quả cuối cùng:

$$\dot{V} = -k_x e_x^2 - \frac{k_\theta}{k_y} \sin^2 e_\theta \leq 0 \quad (19)$$

Vì  $\dot{V} \leq 0$  (Hàm bán xác định âm - Negative Semi-Definite), theo lý thuyết Lyapunov (và kết hợp Bổ đề Barbalat để xử lý triệt để  $e_y$ ), hệ thống sẽ ổn định tiệm cận toàn cục, nghĩa là sai số  $[e_x, e_y, e_\theta]^T \rightarrow 0$  khi  $t \rightarrow \infty$ .

## 4 Điều khiển Động cơ (Low-level): Khâu PID

Để các bánh xe bám theo vận tốc tham chiếu do tầng Backstepping yêu cầu, một bộ điều khiển PID (Proportional-Integral-Derivative) được sử dụng độc lập cho mỗi động cơ.

### 4.1 Luật điều khiển PID

Tín hiệu điều khiển  $u(t)$  được tính dựa trên sai số vận tốc  $e(t) = \omega_{ref}(t) - \omega_{meas}(t)$ :

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (20)$$

Dạng rời rạc được triển khai trên vi điều khiển:

$$u_k = K_p e_k + K_i \sum_{i=0}^k e_i \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t} \quad (21)$$

### 4.2 Chống bão hòa tích phân (Anti-windup)

Trong thực tế, động cơ có giới hạn điện áp (ví dụ  $\pm 12V$ ). Nếu sai số duy trì lâu, khâu tích phân sẽ tăng lên rất lớn (windup), làm hệ thống mất ổn định. Để khắc phục, thành phần  $\sum e_i \Delta t$  được kẹp (clamp) trong một khoảng giới hạn an toàn.

```
1  class WheelPID {
2      private:
3          float Kp = 1.5, Ki = 0.2, Kd = 0.05; // àCn tuning ực ết
4          float error_sum = 0;
5          float last_error = 0;
6          const float dt = 0.02;
7
8      public:
9          float update(float omega_meas, float omega_ref) {
10             float error = omega_ref - omega_meas;
11
12             error_sum += error * dt;
13             error_sum = constrain(error_sum, -50.0, 50.0); // Anti-windup
14
15             float d_error = (error - last_error) / dt;
16             float u_out = Kp * error + Ki * error_sum + Kd * d_error;
17
18             last_error = error;
19             return constrain(u_out, -12.0, 12.0); // Voltage saturation
20         }
21     };
```

Listing 1: Triển khai bộ điều khiển PID cho động cơ bánh xe

## 5 Định vị và Dung hợp Cảm biến (Sensor Fusion)

### 5.1 Tính toán Odometry (Định vị tương đối)

Quy trình tính toán tích phân quãng đường dựa trên số xung encoder:

1. Tính quãng đường bánh trái/phải:  $ds_{L,R} = \frac{2\pi r \cdot \Delta N_{L,R}}{PPR}$ .

2. Quãng đường robot di chuyển:  $ds = \frac{ds_R + ds_L}{2}$ .

3. Độ thay đổi góc hướng:  $\Delta\theta = \frac{ds_R - ds_L}{b}$ .

Cập nhật trạng thái tại bước thời gian  $k + 1$ :

$$x_{k+1} = x_k + ds \cos\left(\theta_k + \frac{\Delta\theta}{2}\right) \quad (22)$$

$$y_{k+1} = y_k + ds \sin\left(\theta_k + \frac{\Delta\theta}{2}\right) \quad (23)$$

$$\theta_{k+1} = \theta_k + \Delta\theta \quad (24)$$

## 5.2 Dung hợp cảm biến với IMU MPU6050

Hạn chế lớn nhất của Odometry là sai số tích lũy do trượt bánh xe. Gyroscope (IMU) không bị ảnh hưởng bởi hiện tượng trượt nhưng lại bị trôi (drift) theo thời gian.

**Phương pháp 1: Complementary Filter.** Góc xoay  $\theta$  được cập nhật đơn giản qua bộ lọc bù:

$$\theta_{k+1} = \alpha(\theta_k + \dot{\theta}_{imu}\Delta t) + (1 - \alpha)\theta_{encoder} \quad (25)$$

Với  $\alpha \in (0.95, 1)$ , phương án này loại trừ nhiễu tần số cao từ encoder và độ trôi tần số thấp của gyroscope.

**Phương pháp 2: Extended Kalman Filter (EKF).** Tuyến tính hóa mô hình phi tuyến thông qua Jacobian  $F_k = \frac{\partial f}{\partial x}$  và  $H_k = \frac{\partial h}{\partial x}$ . EKF dung hợp theo 2 bước:

- *Dự đoán (Encoder)*: Dự đoán trạng thái mới dựa trên dịch chuyển bánh xe.
- *Cập nhật (IMU)*: Hiệu chỉnh góc xoay  $\theta$  sử dụng dữ liệu gyroscope, qua đó thu nhỏ kích thước ma trận hiệp phương sai  $P$ .

## 6 Thực thi Phần cứng trên ESP32

Dưới đây là mã nguồn C++ thực thi trên ESP32. Các cấu trúc dữ liệu đã được bổ sung đầy đủ, và chú thích được viết bằng tiếng Anh để đảm bảo chuẩn hóa lập trình nhưng.

```
1  #include <Arduino.h>
2  #include <cmath>
3
4  // 1. Robot physical and control parameters
5  struct RobotConfig {
6      const double r = 0.05;           // Wheel radius (meters)
7      const double b = 0.5;           // Track width (meters)
8      const double Kx = 2.0;          // Backstepping gain for X-error
9      const double Ky = 5.0;          // Backstepping gain for Y-error
10     const double Kth = 1.5;          // Backstepping gain for Theta-
11     error
12     const double dt = 0.02;          // Sampling time (20ms)
13     };
```

```

14 struct State { double x, y, th; };
15 RobotConfig cfg;
16
17 // 2. Trajectory Generation Struct
18 struct TrajectoryPoint {
19     double x, y, theta, v, omega;
20     double vx, vy, ax, ay; // Internal derivatives
21 };
22
23 // Generate a figure-8 trajectory (Lemniscate of Gerono)
24 TrajectoryPoint getFigure8(double t) {
25     TrajectoryPoint p;
26     double A = 1.2; // Amplitude
27     double w = 0.4; // Orbital frequency
28
29     p.x = A * sin(w * t);
30     p.y = A * sin(w * t) * cos(w * t);
31
32     p.vx = A * w * cos(w * t);
33     p.vy = A * w * (cos(w * t) * cos(w * t) - sin(w * t) * sin(w *
34 t));
35
36     p.theta = atan2(p.vy, p.vx);
37     p.v = sqrt(p.vx * p.vx + p.vy * p.vy);
38     return p;
39 }
40
41 // Generate a circular trajectory
42 TrajectoryPoint generate_circular(double t, double R = 1.0,
43 double w_orbit = 0.5) {
44     TrajectoryPoint p;
45
46     // Position:  $x = R \cos(w \cdot t)$ ,  $y = R \sin(w \cdot t)$ 
47     p.x = R * std::cos(w_orbit * t);
48     p.y = R * std::sin(w_orbit * t);
49
50     // Velocity derivatives:  $v_x = -R \cdot w \sin(w \cdot t)$ ,  $v_y = R \cdot w \cos(w \cdot t)$ 
51     p.vx = -R * w_orbit * std::sin(w_orbit * t);
52     p.vy = R * w_orbit * std::cos(w_orbit * t);
53
54     // Acceleration derivatives:  $a_x = -R \cdot w^2 \cos(w \cdot t)$ ,  $a_y = -R \cdot w^2 \sin(w \cdot t)$ 
55     p.ax = -R * w_orbit * w_orbit * std::cos(w_orbit * t);
56     p.ay = -R * w_orbit * w_orbit * std::sin(w_orbit * t);
57
58     // Desired heading
59     p.theta = std::atan2(p.vy, p.vx);
60
61     // Linear velocity:  $v_d = \sqrt{v_x^2 + v_y^2}$ 
62     p.v = std::sqrt(p.vx * p.vx + p.vy * p.vy);
63 }

```

```

62 // Angular velocity:  $wd = (vx*ay - vy*ax) / v^2$ 
63 p.omega = (p.vx * p.ay - p.vy * p.ax) / (p.v * p.v + 1e-9);
64
65 return p;
66 }
67
68 // 3. Low-level Optimal Control
69 #include <Arduino.h>
70 // Class PID Controller cho ừng bánh xe
71 class WheelPID {
72 private:
73 float Kp, Ki, Kd;
74 float error_sum = 0.0;
75 float last_error = 0.0;
76 float dt = 0.02; // Chu ỳk ấly ẫmu (20ms)
77
78 // ớGii ặhn tích phân (Anti-windup)
79 const float MAX_INTEGRAL = 50.0;
80
81 public:
82 WheelPID(float p, float i, float d) : Kp(p), Ki(i), Kd(d) {}
83
84 float update(float omega_meas, float omega_ref) {
85 // 1. Error Calculation
86 float error = omega_ref - omega_meas;
87
88 // 2. Integral with Anti-windup
89 error_sum += error * dt;
90 if (error_sum > MAX_INTEGRAL) error_sum = MAX_INTEGRAL;
91 if (error_sum < -MAX_INTEGRAL) error_sum = -MAX_INTEGRAL;
92
93 // 3. Derivative
94 float d_error = (error - last_error) / dt;
95
96 // 4. Control signal
97 float u_out = (Kp * error) + (Ki * error_sum) + (Kd *
d_error);
98
99 // Save error
100 last_error = error;
101
102 // Voltage saturation -12V to 12V
103 return constrain(u_out, -12.0, 12.0);
104 }
105
106 // Reset
107 void reset() {
108 error_sum = 0.0;
109 last_error = 0.0;
110 }
111 };

```



```

112
113 // 4. Main Control Task (FreeRTOS Task)
114 void controlTask(void *pv) {
115     TickType_t xLastWakeTime = xTaskGetTickCount();
116
117     //Initialize the PID (Kp, Ki, Kd)
118     WheelPID leftMotorPID(1.5, 0.2, 0.05); // must be tuned
119     WheelPID rightMotorPID(1.5, 0.2, 0.05);
120
121     State current = {0, 0, 0};
122
123     for (;;) {
124         double t = millis() / 1000.0;
125         TrajectoryPoint ref = getFigure8(t); // or generate_circular
126         (t)
127
128         // --- High-level: Backstepping Control ---
129         double dx = ref.x - current.x;
130         double dy = ref.y - current.y;
131         double eth = atan2(sin(ref.theta - current.th), cos(ref.
132         theta - current.th));
133
134         double ex = dx * cos(current.th) + dy * sin(current.th);
135         double ey = -dx * sin(current.th) + dy * cos(current.th);
136
137         double v_cmd = ref.v * cos(eth) + cfg.Kx * ex;
138         double w_cmd = ref.omega + ref.v * (cfg.Ky * ey + cfg.Kth *
139         sin(eth));
140
141         // --- Inverse Kinematics ---
142         double wL_ref = (v_cmd - (cfg.b/2)*w_cmd) / cfg.r;
143         double wR_ref = (v_cmd + (cfg.b/2)*w_cmd) / cfg.r;
144
145         // --- Low-level: Execute LQG ---
146         // --- Low-level: Execute PID ---
147         // Assume getEncoderVelocityL() to measure angular speed (
148         rad/s)
149
150         float omega_meas_L = getEncoderVelocityL();
151         float omega_meas_R = getEncoderVelocityR();
152
153         float uL = leftMotorPID.update(omega_meas_L, wL_ref);
154         float uR = rightMotorPID.update(omega_meas_R, wR_ref);
155
156         vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(20));
157     }
158 }
159
160 void setup() {
161     xTaskCreate(controlTask, "ControlTask", 4096, NULL, 10, NULL);
162 }

```

159  
160  
161

```
void loop() {}
```

Listing 2: ESP32 implementation for Backstepping, LQG, and Trajectory Generation

## 7 Kết luận và Lưu ý Triển khai

Các lưu ý quan trọng trong quá trình lập trình phần cứng:

- **Định thời (Timing):** Bắt buộc sử dụng RTOS (cụ thể là `vTaskDelayUntil` trong FreeRTOS) để giữ chu kỳ lấy mẫu  $dt$  ổn định ( $20ms$ ). Bất kỳ độ trễ (jitter) nào cũng sẽ làm sai lệch bộ lọc Kalman và Odometry.
- **Chuẩn hóa đơn vị đo đạc:** Tất cả thông số phải sử dụng chuẩn SI (mét, radian, giây). Xung encoder (ticks) cần được chuyển đổi sang  $rad/s$  trước khi đưa vào hàm tính toán của động cơ.
- **Bảo hòa ngõ ra (Actuator Saturation):** Tín hiệu điều khiển  $u_{volt}$  phải luôn được kẹp giới hạn theo ngưỡng điện áp nguồn của driver động cơ, phòng tránh hiện tượng windup:

$$u_{out} = \max(-12, \min(12, u)) \quad (26)$$

- **Hiệu chuẩn Cảm biến (Calibration):** Trạng thái đứng yên đầu tiên sau khi khởi động phải được dùng để xác định sai số tĩnh (offset) cho Gyroscope nhằm chống hiện tượng drift hệ thống.